CAPSTONE PROJECT · TECHNICAL DOCUMENTATION

SmartToll

Toll route planner with RFID balance tracking for motorists in Luzon

Holy Angel University

October 21, 2026

Contents

1. **System architecture** — overview, toll computation, saving a trip, GPS tracking, deployment, security

2. **Database** — entity-relationship diagram, every table and column

3. **REST API** — all 62 endpoints

4. **Automated tests** — 52 of 52 passed, end-to-end checks

About this document

Technical documents for the SmartToll capstone project (Holy Angel University). The diagrams are written in Mermaid, which GitHub draws directly when you open these files there.

Document	Contents
ARCHITECTURE.md	System overview, how a toll is computed, saving a trip and the RFID deduction, GPS trip tracking, deployment, security measures
DATABASE.md	Entity-relationship diagram (ERD) and every table and column, generated from the live schema
API.md	Every REST endpoint (public, motorist, administrator), generated from the route list
TESTING.md	Automated test results (52 tests) and the end-to-end checks

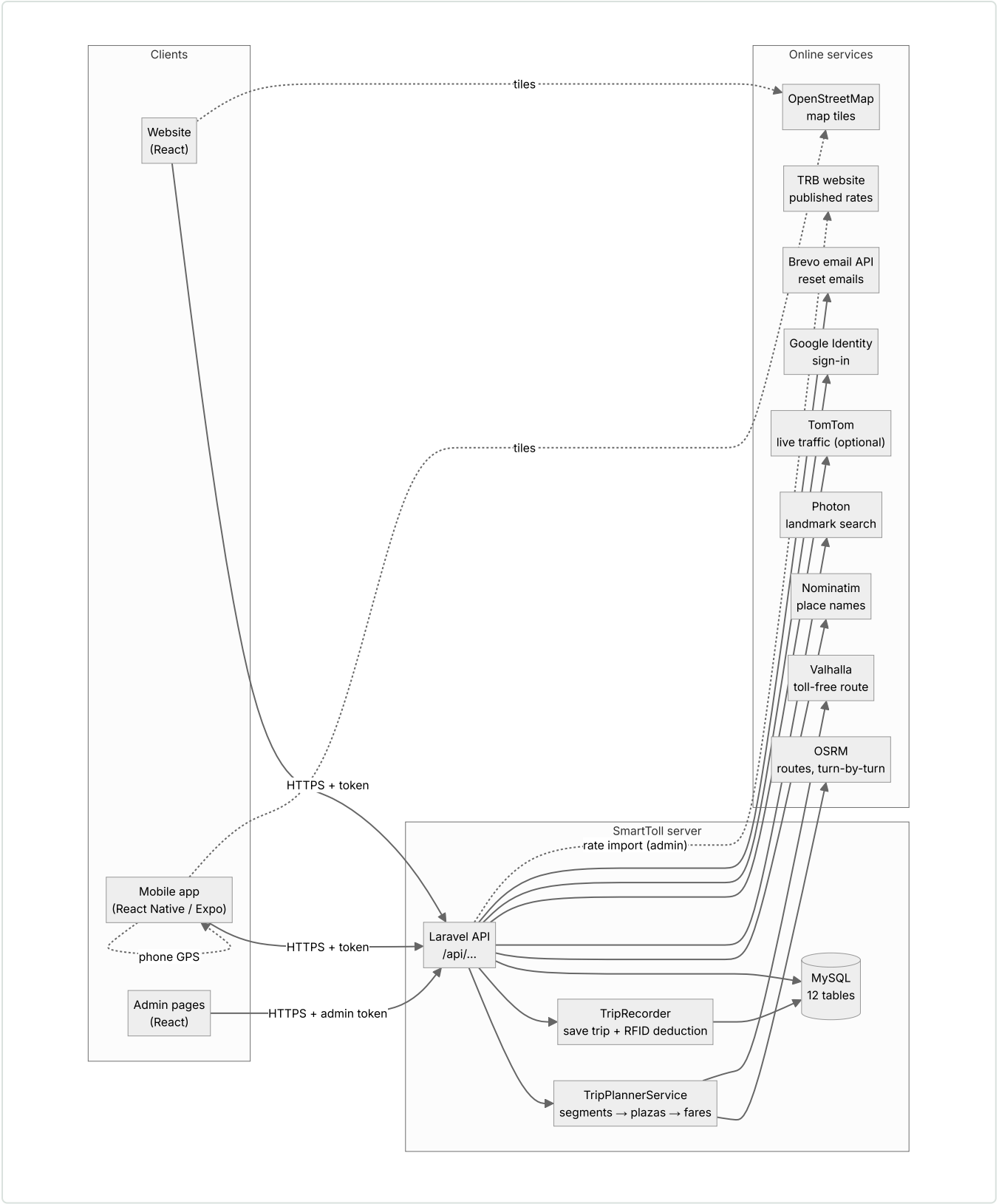
For installing and running SmartToll, see the main README in the project repository.

SmartToll system architecture

SmartToll is a toll route planner with RFID balance tracking for motorists in Luzon. It has three parts that share one server:

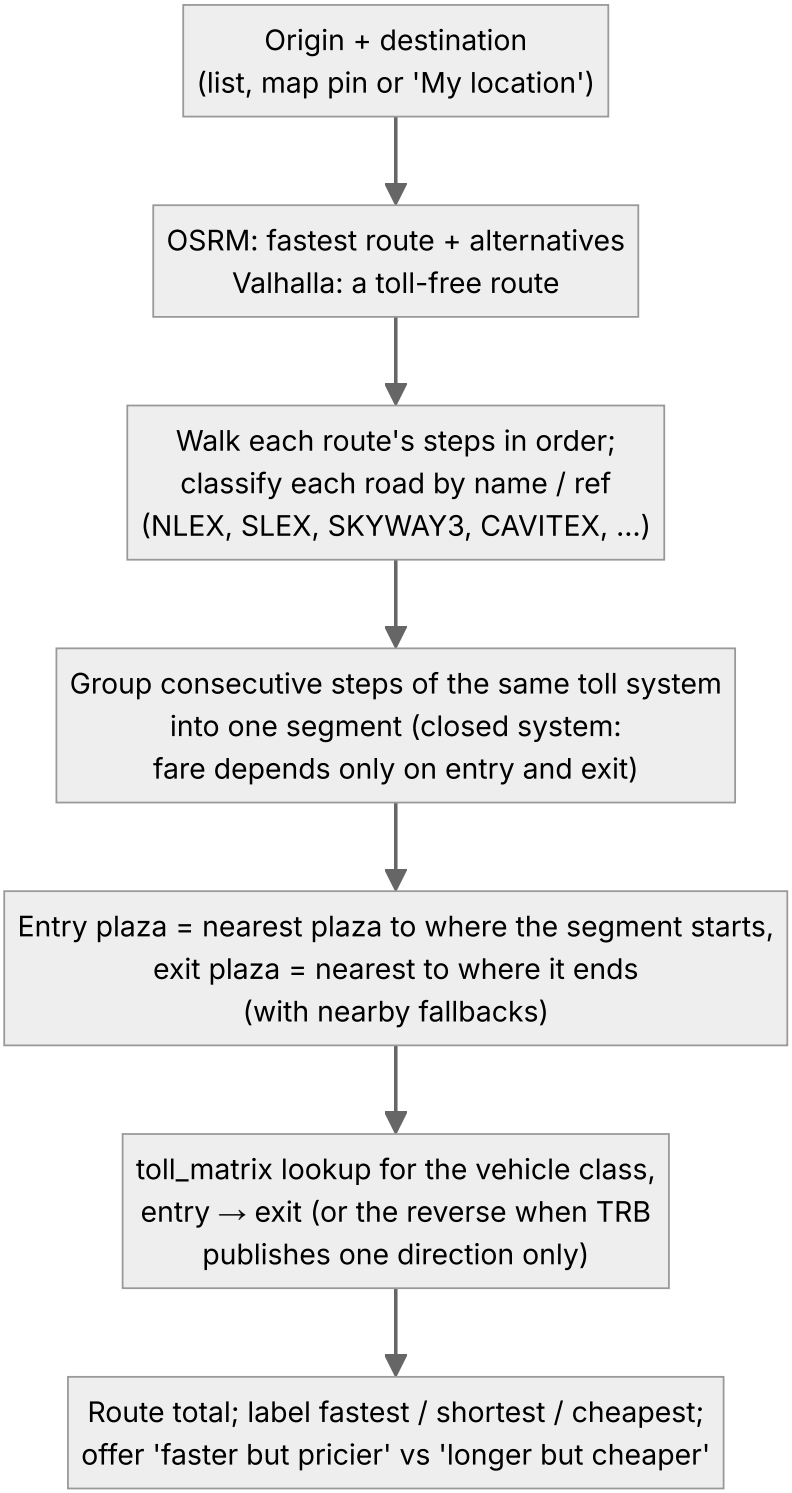
Part	Built with	Used by
Website	React 18, React Router, Leaflet (map), Vite	Motorists (and visitors, who can plan without an account) and administrators
Mobile app	React Native 0.86 with Expo SDK 57, Expo Router, expo-location, expo-speech, Leaflet in a WebView	Motorists on Android / iPhone
API	Laravel 12 (PHP 8.2+), Laravel Sanctum (login tokens), MySQL / MariaDB	Both apps; all toll computation happens here

System overview



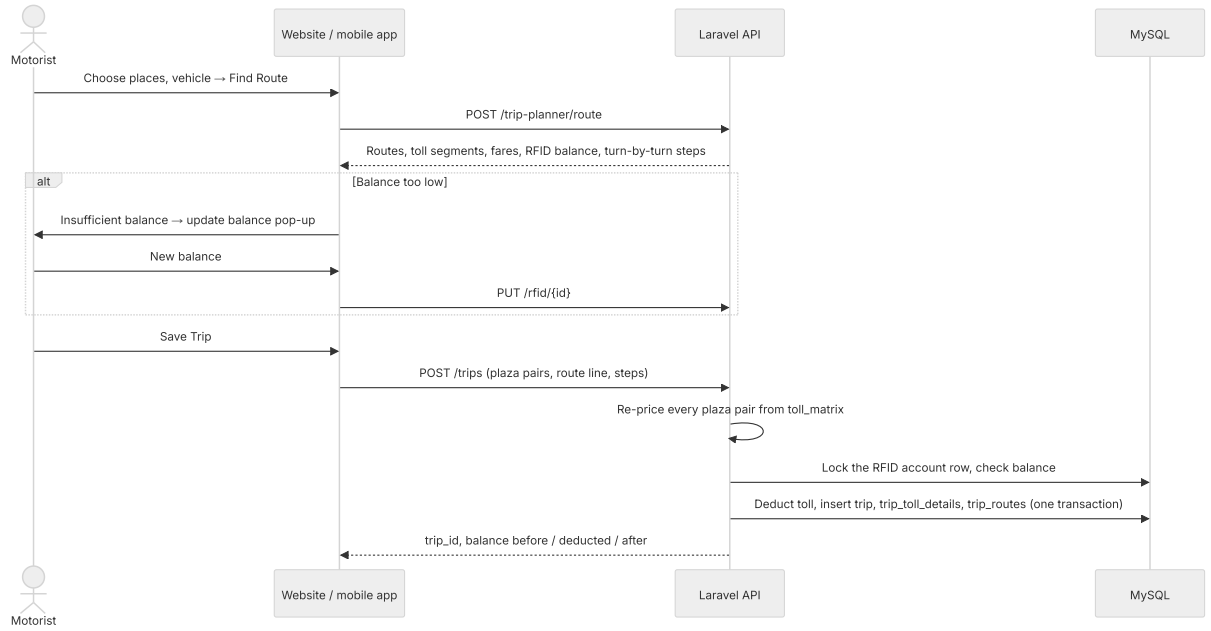
The apps never compute fares: they send places (or plaza pairs) and the API prices them from the TRB toll matrix. When a trip is saved, the API prices it again, so a modified app can't change what is charged.

How a toll is computed

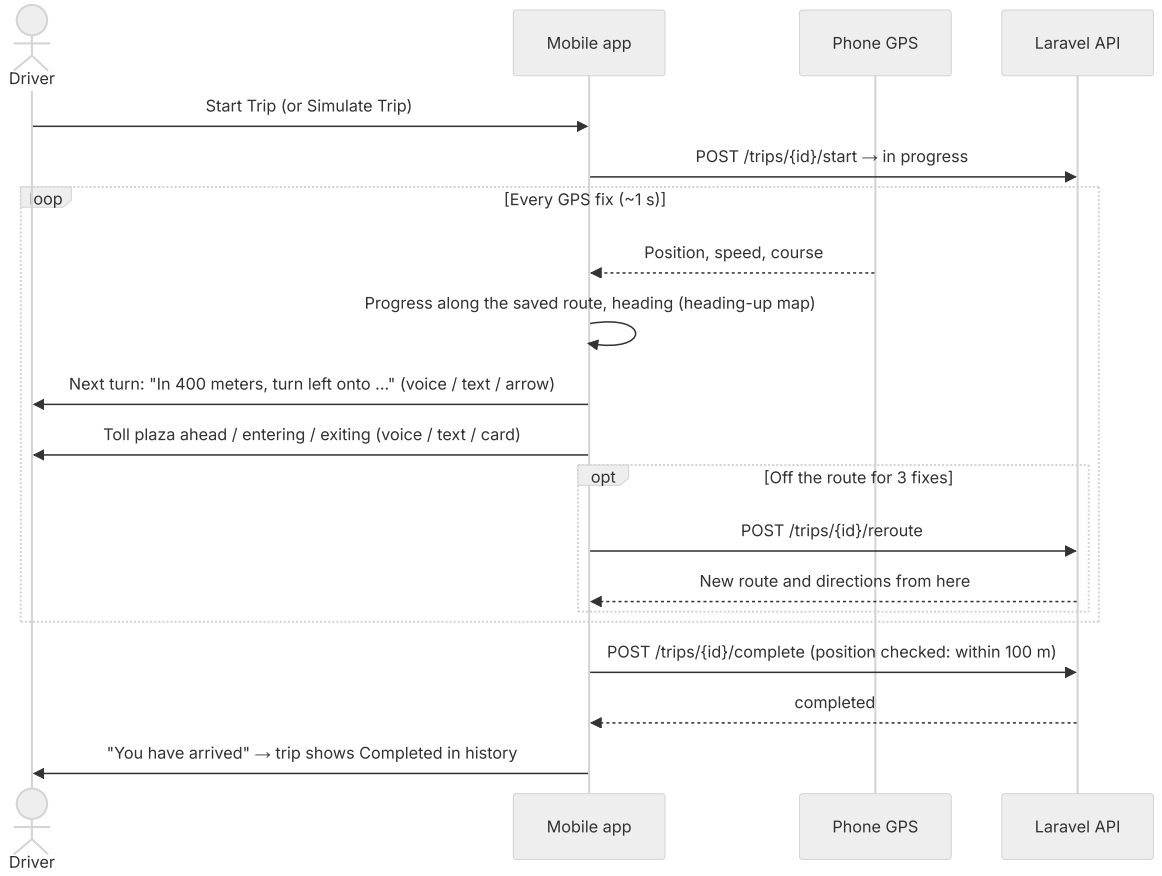


Special cases handled: the elevated Skyway is priced as its own "SKY" plaza; Skyway Stage 3 is split from the older Skyway at Buendia; a short brush with an expressway at a single plaza is free; tolled roads without loaded rates are reported instead of priced at zero.

Saving a trip and paying from the RFID balance

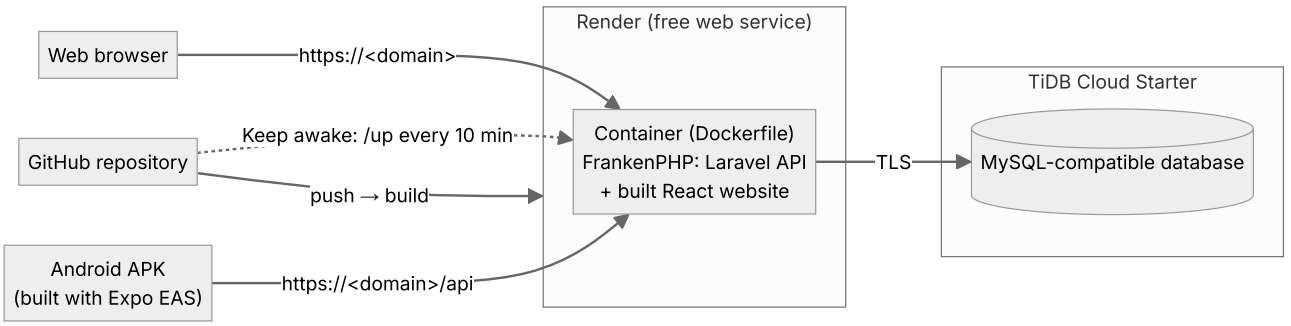


Following a trip with GPS (mobile app)



The toll is not charged again while driving: it was deducted when the trip was saved. With "Allow all the time" location, the phone keeps following the trip when it is locked or another app is open (Android shows a notification while it does).

Deployment



On start, the container sets up the database by itself (`php artisan smarttoll:install`: loads `smarttoll.sql` into an empty database, then runs newer migrations). Debug mode is off in production; https ends at Render's proxy, which Laravel trusts. The database connection is encrypted (TiDB Cloud accepts TLS only; `MYSQL_ATTR_SSL_CA` points at the image's CA bundle). Render's free service sleeps after 15 idle minutes, so a scheduled GitHub workflow opens `/up` every 10 minutes.

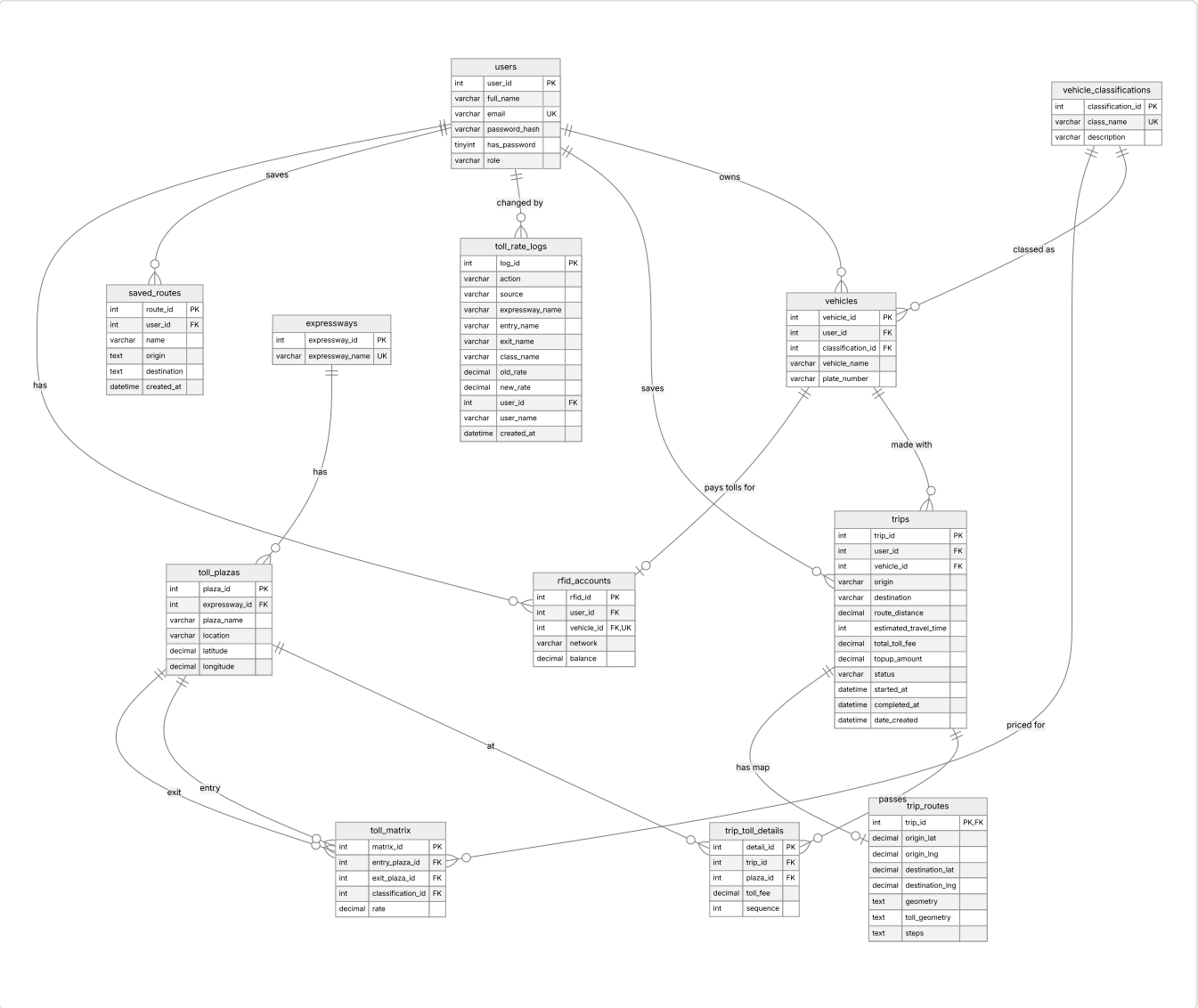
Security measures

- **Passwords:** stored as bcrypt hashes; password-reset links are single-use and expire.
- **Logins:** Sanctum bearer tokens; administrators have a separate role checked on every admin request (`EnsureAdmin`).
- **Ownership:** every vehicle, RFID account, trip and saved route is checked against the logged-in user.
- **Server-side pricing:** fares sent by an app are ignored; trips are re-priced from the TRB matrix when saved.
- **Consistent balances:** the RFID balance is locked during a save, so two saves can't both spend the same balance.
- **Google sign-in:** the ID token is verified with Google and must be issued for SmartToll's own client ID, for a verified email.
- **Rate limits:** login-related, routing and import endpoints are throttled (see API.md).
- **Arrival check:** a trip can only be completed within 100 m of its saved destination (the app marks it on reaching the pin: within 50 m, or at the very end of the route).

SmartToll database

Generated from the live MySQL schema on October 13, 2026. 12 application tables (plus 3 Laravel framework tables), 15 foreign keys.

Entity-relationship diagram



Read `A |⊢-o{ B` as "one A has zero or more B"; `|⊢-o|` as "zero or one".

Tables

`users`

Motorists and administrators (role). Passwords are stored as bcrypt hashes.

Column	Type	Null	Key
`user_id`	int(11)	no	primary key
`full_name`	varchar(100)	no	
`email`	varchar(100)	no	unique
`password_hash`	varchar(255)	no	
`has_password`	tinyint(1)	no	default 1; 0 for accounts made with Google until a password is set
`role`	varchar(20)	no	

`vehicles`

A motorist's vehicles; the classification sets the toll rate.

Column	Type	Null	Key
`vehicle_id`	int(11)	no	primary key
`user_id`	int(11)	no	→ `users.user_id`
`classification_id`	int(11)	no	→ `vehicle_classifications.classification_id`
`vehicle_name`	varchar(100)	no	
`plate_number`	varchar(15)	yes	

`vehicle_classifications`

TRB vehicle classes (Class 1, 2, 3) with their descriptions.

Column	Type	Null	Key
`classification_id`	int(11)	no	primary key
`class_name`	varchar(20)	no	unique
`description`	varchar(255)	no	

`rfid_accounts`

One Easytrip or Autosweep account per vehicle, with its recorded balance.

Column	Type	Null	Key
`rfid_id`	int(11)	no	primary key
`user_id`	int(11)	no	→ `users.user_id`
`vehicle_id`	int(11)	no	unique, → `vehicles.vehicle_id`
`network`	varchar(20)	no	
`balance`	decimal(10,2)	no	

`trips`

Saved trips: route summary, total toll, and progress (planned → in progress → completed).

Column	Type	Null	Key
`trip_id`	int(11)	no	primary key
`user_id`	int(11)	no	→ `users.user_id`
`vehicle_id`	int(11)	no	→ `vehicles.vehicle_id`
`origin`	varchar(150)	no	
`destination`	varchar(150)	no	
`route_distance`	decimal(8,2)	no	
`estimated_travel_time`	int(11)	no	
`total_toll_fee`	decimal(10,2)	no	
`topup_amount`	decimal(10,2)	no	
`status`	varchar(20)	no	
`started_at`	datetime	yes	
`completed_at`	datetime	yes	
`date_created`	datetime	no	

`trip_toll_details`

Each trip's toll plazas in order: the entry plaza at ₱0, then the exit plaza with the fare.

Column	Type	Null	Key
`detail_id`	int(11)	no	primary key
`trip_id`	int(11)	no	→ `trips.trip_id`
`plaza_id`	int(11)	no	→ `toll_plazas.plaza_id`
`toll_fee`	decimal(10,2)	no	
`sequence`	int(11)	no	

`trip_routes`

Each trip's map: origin/destination coordinates, route line, toll stretches and turn-by-turn steps.

Column	Type	Null	Key
`trip_id`	int(11)	no	primary key, → `trips.trip_id`
`origin_lat`	decimal(10,7)	no	
`origin_lng`	decimal(10,7)	no	
`destination_lat`	decimal(10,7)	no	

Column	Type	Null	Key
`destination_lng`	decimal(10,7)	no	
`geometry`	longtext	no	
`toll_geometry`	longtext	no	
`steps`	longtext	yes	

`saved_routes`

A motorist's saved routes ("Home → Office").

Column	Type	Null	Key
`route_id`	int(10) unsigned	no	primary key
`user_id`	int(11)	no	→ `users.user_id`
`name`	varchar(100)	no	
`origin`	text	no	
`destination`	text	no	
`created_at`	datetime	no	

`expressways`

Toll expressways (NLEX / SCTEX, SLEX, TPLEX, CAVITEX, …).

Column	Type	Null	Key
`expressway_id`	int(11)	no	primary key
`expressway_name`	varchar(100)	no	unique

`toll_plazas`

Toll plazas of each expressway, with map coordinates.

Column	Type	Null	Key
`plaza_id`	int(11)	no	primary key
`expressway_id`	int(11)	no	→ `expressways.expressway_id`
`plaza_name`	varchar(100)	no	
`location`	varchar(150)	no	
`latitude`	decimal(10,7)	no	
`longitude`	decimal(10,7)	no	

`toll_matrix`

Published TRB fare for each entry plaza → exit plaza pair and vehicle class.

Column	Type	Null	Key
`matrix_id`	int(11)	no	primary key
`entry_plaza_id`	int(11)	no	→ `toll_plazas.plaza_id`
`exit_plaza_id`	int(11)	no	→ `toll_plazas.plaza_id`
`classification_id`	int(11)	no	→ `vehicle_classifications.classification_id`
`rate`	decimal(10,2)	no	

`toll_rate_logs`

Change log of toll rates: old → new, who, when and the source (edit, CSV, TRB import).

Column	Type	Null	Key
`log_id`	int(10) unsigned	no	primary key
`action`	varchar(10)	no	
`source`	varchar(20)	no	
`expressway_name`	varchar(100)	no	
`entry_name`	varchar(100)	no	
`exit_name`	varchar(100)	no	
`class_name`	varchar(20)	no	
`old_rate`	decimal(10,2)	yes	
`new_rate`	decimal(10,2)	yes	
`user_id`	int(11)	yes	→ `users.user_id`
`user_name`	varchar(100)	yes	
`created_at`	datetime	no	

`migrations`

Laravel: which database migrations have run.

Column	Type	Null	Key
`id`	int(10) unsigned	no	primary key
`migration`	varchar(255)	no	
`batch`	int(11)	no	

`password_reset_tokens`

Laravel: one-time tokens for password-reset emails.

Column	Type	Null	Key
`email`	varchar(255)	no	primary key
`token`	varchar(255)	no	
`created_at`	timestamp	yes	

`personal_access_tokens`

Laravel Sanctum: login tokens of the website and the mobile app.

Column	Type	Null	Key
`id`	bigint(20) unsigned	no	primary key
`tokenable_type`	varchar(255)	no	
`tokenable_id`	bigint(20) unsigned	no	
`name`	varchar(255)	no	
`token`	varchar(64)	no	unique
`abilities`	text	yes	
`last_used_at`	timestamp	yes	
`expires_at`	timestamp	yes	
`created_at`	timestamp	yes	
`updated_at`	timestamp	yes	

SmartToll REST API

Generated from `php artisan route:list` on October 13, 2026: 62 endpoints. Base URL: `https://<server>/api`. Requests and responses are JSON; logged-in requests send `Authorization: Bearer <token>`. The website and the mobile app use this same API; all toll computation happens here.

Public

Method	Endpoint	What it does
POST	`/api/admin/login`	Administrator log in
POST	`/api/auth/google`	Log in with a Google ID token *(max 10 per 1 min)*
GET	`/api/auth/google/config`	Google client ID for the sign-in button
POST	`/api/forgot-password`	Email a password-reset link *(max 6 per 1 min)*
POST	`/api/login`	Motorist log in (returns a token)
POST	`/api/register`	Create a motorist account
POST	`/api/reset-password`	Set a new password with the emailed token *(max 6 per 1 min)*
GET	`/api/trip-planner/locations`	Supported places and toll plazas, vehicle classes *(max 60 per 1 min)*
GET	`/api/trip-planner/place-name`	Place name of a map pin *(max 20 per 1 min)*
GET	`/api/trip-planner/search`	Landmarks and places in Luzon matching `q` (ids `osm:…` usable as origin / destination) *(max 60 per 1 min)*
GET	`/api/traffic/status`	Whether live traffic is set up (`enabled`) *(max 60 per 1 min)*
GET	`/api/traffic/tiles/{z}/{x}/{y}.png`	Live traffic map tile (TomTom, through the server so the key stays private) *(max 600 per 1 min)*
POST	`/api/trip-planner/route`	Routes with toll segments and fares (fastest / shortest / cheapest) *(max 20 per 1 min)*

Motorist (requires login token)

Method	Endpoint	What it does
GET	`/api/dashboard`	Dashboard figures, balances, recent trips
POST	`/api/logout`	Log out (revokes the token)
GET	`/api/profile`	The signed-in user (incl. `has_password`)
PUT	`/api/profile`	Update name and email
PUT	`/api/profile/password`	Change password; Google accounts without one just set it (no `current_password`) *(max 10 per 1 min)*

Method	Endpoint	What it does
GET	`/api/rfid`	List RFID accounts
POST	`/api/rfid`	Link an RFID account to a vehicle
DELETE	`/api/rfid/{rfid}`	Remove an RFID account
GET	`/api/rfid/{rfid}`	One RFID account
PUT	PATCH	`/api/rfid/{rfid}`
GET	`/api/saved-routes`	List saved routes
POST	`/api/saved-routes`	Save a route
DELETE	`/api/saved-routes/{id}`	Delete a saved route
GET	`/api/trips`	Trip history
POST	`/api/trips`	Save a trip: re-prices it and deducts the toll from the RFID balance
DELETE	`/api/trips/{id}`	Delete a trip
GET	`/api/trips/{id}`	One trip with toll segments, map and directions
POST	`/api/trips/{id}/complete`	Arrived: mark completed (position checked)
POST	`/api/trips/{id}/reroute`	New directions after a wrong turn *(max 10 per 1 min)*
POST	`/api/trips/{id}/traffic`	Live traffic on the road ahead: time left, incidents, and a faster route when traffic is heavy *(max 20 per 1 min)*
POST	`/api/trips/{id}/start`	Start following the trip with GPS (in progress)
GET	`/api/vehicles`	List vehicles and classifications
POST	`/api/vehicles`	Add a vehicle
DELETE	`/api/vehicles/{vehicle}`	Delete a vehicle (not if it has trips)
GET	`/api/vehicles/{vehicle}`	One vehicle
PUT	PATCH	`/api/vehicles/{vehicle}`

Administration (admin token)

Method	Endpoint	What it does
GET	`/api/admin/expressways`	List expressways
POST	`/api/admin/expressways`	Add an expressway
DELETE	`/api/admin/expressways/{id}`	Delete an expressway (and its plazas and rates)
PUT	`/api/admin/expressways/{id}`	Edit an expressway
POST	`/api/admin/rate-import/csv/apply`	Apply a previewed CSV import (logged)
POST	`/api/admin/rate-import/csv/preview`	Preview a CSV rate import (changes nothing)
POST	`/api/admin/rate-import/trb/apply`	Apply a previewed TRB import (logged)

Method	Endpoint	What it does
POST	`/api/admin/rate-import/trb/preview`	Preview rates from a TRB page (changes nothing): an uploaded page saved from the browser, whose expressway is read from the file, or a download / the server's copy *(max 10 per 1 min)*
GET	`/api/admin/rate-log`	Toll rate change log
GET	`/api/admin/summary`	Dashboard counts and the Needs Attention list
GET	`/api/admin/toll-matrix`	Browse toll rates (filters, search, pages)
POST	`/api/admin/toll-matrix`	Add a toll rate (logged)
GET	`/api/admin/toll-matrix/export`	Download the rates as CSV
DELETE	`/api/admin/toll-matrix/{id}`	Delete a toll rate (logged)
PUT	`/api/admin/toll-matrix/{id}`	Edit a toll rate (logged)
GET	`/api/admin/toll-plazas`	List toll plazas
POST	`/api/admin/toll-plazas`	Add a toll plaza
DELETE	`/api/admin/toll-plazas/{id}`	Delete a toll plaza (not if trips use it)
PUT	`/api/admin/toll-plazas/{id}`	Edit a toll plaza (name, map location)
GET	`/api/admin/vehicle-classes`	List vehicle classes
POST	`/api/admin/vehicle-classes`	Add a vehicle class
DELETE	`/api/admin/vehicle-classes/{id}`	Delete a vehicle class (not while in use)
PUT	`/api/admin/vehicle-classes/{id}`	Edit a vehicle class

SmartToll automated tests

Run on October 13, 2026 with `php artisan test` *(PHPUnit 11, PHP 8.2, MariaDB 10.4):* ***52 of 52 passed****, 239 assertions.* (Updated October 21, 2026 with the TRB page upload.)

The tests use a separate database (`smarttoll_test`) built from `database/smarttoll.sql` ; every test runs in a transaction that is rolled back. Outside services are replaced by recorded answers (a real OSRM route for Quezon City → San Fernando via NLEX, Google token checks), so the results are repeatable and run offline.

Area	Tests	Passed
Accounts: saved routes, Google sign-in, passwords	6	6
Administration: access, rate rules, change log, CSV/TRB import	11	11
Toll computation: TRB fares, segment detection, route comparison	12	12
Trips and RFID: planning, landmarks, map pins, live traffic, saving, deduction, tracking, directions	23	23
Total	**52**	**52**

Accounts: saved routes, Google sign-in, passwords

- ✓ A motorist can save list and delete routes
- ✓ Saved routes are private
- ✓ Google sign in creates a motorist once
- ✓ A google account sets a password without a current one
- ✓ Google tokens for another app are rejected
- ✓ Google sign in is refused for unverified emails and admins

Administration: access, rate rules, change log, CSV/TRB import

- ✓ Motorists and visitors cannot use the admin api
- ✓ Duplicate and cross expressway rates are rejected
- ✓ Editing a rate is written to the change log
- ✓ Deleting a plaza logs each rate removed with it
- ✓ A classification in use cannot be deleted
- ✓ A plaza used by a saved trip cannot be deleted
- ✓ Csv preview changes nothing and apply updates and logs
- ✓ Csv errors are reported by line
- ✓ Trb import from the saved page restores an edited rate
- ✓ An uploaded trb page is recognized and previewed
- ✓ Rates export as csv

Toll computation: TRB fares, segment detection, route comparison

- [x] Class number is read from the class name
- [x] Toll system key comes from the expressway name
- [x] Fare is looked up from the trb matrix per vehicle class
- [x] A fare stored one way also prices the opposite direction
- [x] No fare between plazas of different expressways
- [x] A route on nlex is split into one segment with the right entry and exit
- [x] The route is priced for the vehicle class
- [x] Planning labels the fastest shortest and cheapest route
- [x] A short brush with an expressway at one plaza is not tolled
- [x] A nearby plaza is used when the nearest one has no fare
- [x] The cavitex calax link road counts as cavitex
- [x] A fast but pricier route is offered against a longer cheaper one

Trips and RFID: planning, landmarks, live traffic, saving, deduction, tracking, directions

- [x] Visitors can plan a trip by vehicle class
- [x] A map pin is named even when nominatim turns the server away
- [x] A map pin without any name service falls back to its coordinates
- [x] Landmarks can be searched and planned from
- [x] Without a traffic key routes keep their usual times
- [x] Live traffic sets the travel time but never the toll
- [x] A followed trip hears about incidents on the road ahead only
- [x] Heavy traffic ahead offers a faster route without touching the paid toll
- [x] A landmark not in the cache is looked up and unknown ids are refused
- [x] Visitors cannot use a saved vehicle or save trips
- [x] A motorist planning with their vehicle gets their rfid account
- [x] Planning with someone elses vehicle is refused
- [x] Saving a trip prices it on the server and deducts the toll
- [x] Saving with an insufficient balance is refused and nothing changes
- [x] A vehicle without an rfid account saves without a deduction
- [x] A plaza pair across expressways cannot be saved
- [x] Saved trips keep what is needed to plan them again
- [x] A saved trip is tracked from planned to completed
- [x] Planned routes carry turn by turn directions that are saved with the trip
- [x] A motorist off the route gets new directions to the destination
- [x] A vehicle with trip history cannot be deleted
- [x] Only the owner can track a trip
- [x] Motorists only see their own trips

Manual and end-to-end checks

Besides the automated tests, the website and the mobile app (Expo web build at phone size) were exercised end to end in Chrome during development: registering, adding vehicles and RFID accounts, planning and saving Quezon City → San Fernando (₱266 deducted), GPS trip tracking in simulation (37 spoken directions, toll entry/exit warnings, arrival marks the trip completed), heading-up map rotation, light and dark mode, and map smoothness with the CPU slowed 4× (full map refreshes cut from 154/s to 2.8/s). Later checks: landmark search on both apps (SM North EDSA → Holy Angel University planned with real fares); dynamic zoom during a simulated trip (zooms in before turns and back out after, never faster than about 1 zoom level a second, still 2.8 full refreshes/s with the CPU slowed 4×); and the landscape trip layout, rotating portrait ↔ landscape mid-trip without the trip resetting (distance left kept counting down). Live traffic was checked end to end against a local stand-in for TomTom (no real key yet): route options showed "1 h 30 min now · +25 min traffic" on both apps, the simulated trip announced an accident on E1 ahead and stretched the arrival time, and the traffic map layer showed on the website and the app (turning with heading-up).

Move to free hosting (October 20, 2026). The website and API moved from Railway to Render, and the database to TiDB Cloud. Before the switch, the database setup (`smarttoll:install`) ran on TiDB without errors. Every table was then copied and compared with the old database: same row counts in all 15 tables, the same RFID balance total (₱13,705.00) and toll fee total (₱4,971.00), and identical checksums for users, vehicles, RFID accounts, trips, toll details, saved routes and login tokens. On the live site, the health check, website, place list, login check and live-traffic status all answered, a guest plan from Quezon City to San Fernando returned the NLEX route at ₱266 and a toll-free alternative, and Google sign-in loaded once the new address was authorized.